**Университет ИТМО**

Факультет программной инженерии и компьютерной техники

# Лабораторная работа №1

«Интерфейсы ввода-вывода общего назначения (GPIO)»
Дисциплина: Проектирование вычислительных систем

Вариант: **1. «Светофор»**

**Выполнили** студенты группы P3410:
Нестеров Владислав Алексеевич
Джохадзе Анна Бекаевна

**Преподаватель:**
Пинкевич Василий Юрьевич

Санкт-Петербург, 2026

# 1. Задание

## 1.1. Прикладная программа. Вариант 1 «Светофор»

Необходимо имитировать светофор пешеходного перехода. Цвета циклически переключаются в порядке: красный, зелёный, мигающий зелёный, жёлтый и снова красный. Обычный период горения красного должен быть в четыре раза больше периода горения зелёного.

Нажатие кнопки во время красного, мигающего зелёного или жёлтого запоминает запрос на скорейшее включение зелёного. Последовательность цветов сохраняется, но ближайший период красного сокращается до четверти обычной длительности. Если к моменту нажатия красный уже горит не менее четверти периода, программа сразу переходит к зелёному. Следующий красный снова имеет обычную длительность, если нового запроса не поступило. Нажатие во время постоянного зелёного не изменяет цикл.

Драйверы светодиодов и кнопки должны быть универсальными, не содержать прикладной логики и блокирующих ожиданий. Для кнопки требуется программное подавление дребезга. Опрос кнопки и управление временными интервалами выполняются кооперативно. Операционная система реального времени, прерывания GPIO и дополнительные таймеры не используются; допускается системный таймер SysTick.

## 1.2. Собственный драйвер GPIO

Вторая часть выполнена по **способу 1**: разработан собственный драйвер GPIO без использования функций HAL и LL для управления портами. Драйвер должен поддерживать настройку и чтение конфигурации выводов, чтение и запись одного вывода и группы выводов, а также повторную настройку режима с предотвращением некорректных кратковременных уровней при переключении на выход.

Допускается использование типа `GPIO_TypeDef` и определений регистров производителя. Структура конфигурации и функции работы с GPIO реализуются самостоятельно. Поддержка прерываний GPIO не требуется.

# 2. Аппаратная конфигурация

Программа предназначена для стенда SDK-1.1M с микроконтроллером STM32F427VIT6. Используются кнопка боковой панели, отдельный зелёный светодиод и двухцветный красно-жёлтый светодиод. Соответствие выводов и сигналов приведено в Таблица 1.

| Элемент | Вывод | Режим и активный уровень |
|---|---|---|
| Кнопка | PC15 | Вход GPIO; нажатие соответствует логическому 0 |
| Зелёный светодиод | PD13 | Выход push-pull; включение при логической 1 |
| Жёлтый цвет | PD14, PD15 | PD14 = 1, PD15 = 0 |
| Красный цвет | PD14, PD15 | PD14 = 0, PD15 = 1 |
| Выключение двухцветного светодиода | PD14, PD15 | В программе оба вывода равны 0 |

Таблица 1. Назначение выводов стенда

Красный и жёлтый цвета реализованы встречным включением кристаллов двухвыводного светодиода. Одновременный запрос этих цветов обрабатывается драйвером как выключение двухцветного индикатора. Маска PD13-PD15 записывается одной операцией, поэтому переключение цветов не требует последовательного изменения отдельных выводов.

В проекте STM32CubeIDE `light` источник системной частоты - внутренний генератор HSI 16 МГц, PLL отключён, делители AHB, APB1 и APB2 равны единице. Именно эта конфигурация находится в текущем `SystemClock_Config()` проекта `light`. Внешний кварц 25 МГц в данной конфигурации не задействован. Счётчик времени `HAL_GetTick()` использует миллисекундный SysTick. Генератор LSE не включается, поскольку вывод PC15 занят кнопкой.

# 3. Архитектура программы

## 3.1. Разделение на модули

Программа разделена на прикладную часть, обработку событий и аппаратные драйверы. Конкретные номера портов и выводов скрыты в модуле стенда. Связи компонентов показаны на Рис. 1.

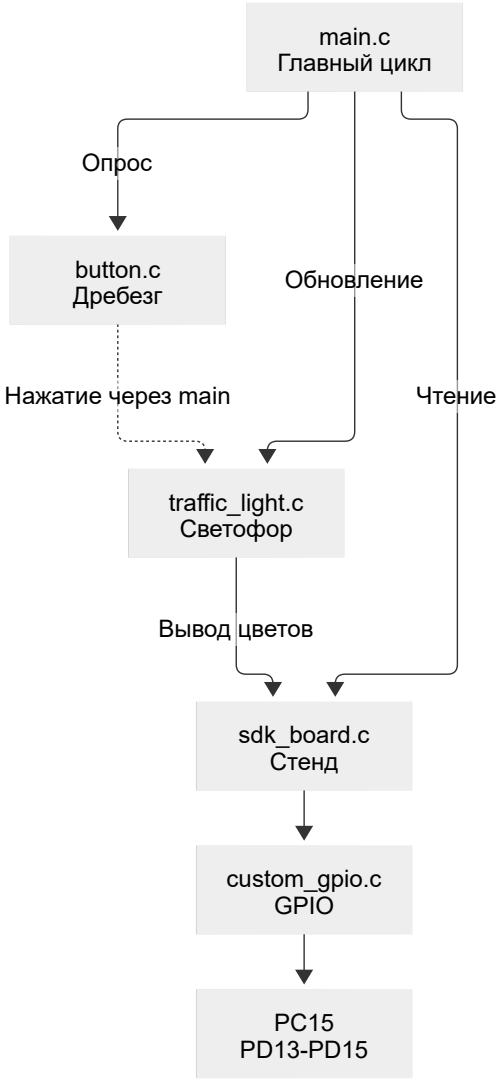



Рис. 1. Связи основных модулей программы

Назначение файлов приведено в Таблица 2. Модули `button` и `traffic_light` не зависят от заголовков STM32.

| **Модуль** | **Назначение** |
|---|---|
| `main.c` | Инициализация, получение времени и последовательный вызов обработчиков |
| `button.c/.h` | Подавление дребезга; события нажатия и отпускания; длительность удержания |
| `traffic_light.c/.h` | Конечный автомат светофора, сроки переключений и отложенный запрос |
| `sdk_board.c/.h` | Настройка выводов стенда, инверсия уровня кнопки и преобразование цветов в маску GPIO |
| `custom_gpio.c/.h` | Собственные типы конфигурации и операции с регистрами GPIO |

Таблица 2. Ответственность программных модулей

Связь автомата с аппаратурой выполняется через функцию обратного вызова `TrafficLightOutput`. Автомат передаёт логические признаки цветов, а функция `set_traffic_lights()` преобразует их в маску `SDK_LED_*`. Поэтому прикладной модуль не обращается к GPIO напрямую.

### 3.2. Кооперативный главный цикл

Каждая итерация считывает время один раз, опрашивает кнопку, выполняет подавление дребезга и передаёт подтверждённое событие автомату. Затем вызывается обновление светофора. Схема прикладного алгоритма приведена на Рис. 2.

**Ожидание реализовано проверкой времени, а не задержкой.** Если срок не наступил, обработчик сразу возвращает управление. Программа продолжает опрашивать кнопку во время всех световых фаз. `HAL_Delay()` и циклы ожидания нажатия в драйверах отсутствуют.

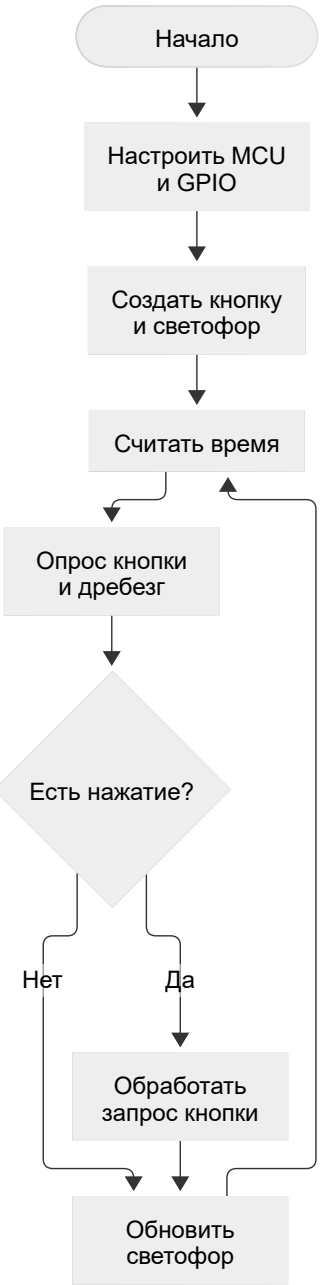



Рис. 2. Блок-схема кооперативного главного цикла

Функция `traffic_light_update()` обрабатывает наступившие сроки без ожидания, выполняя не более 16 шагов за вызов. Сроки обычных переходов отсчитываются от ранее запланированного момента, чтобы не накапливать задержки опроса.

# 4. Прикладной алгоритм

## 4.1. Состояния и временные интервалы

Конечный автомат содержит четыре состояния: `RED`, `GREEN`, `BLINKING_GREEN` и `YELLOW`. После инициализации включается красный. Параметры заданы константами в `traffic_light.h` и приведены в Таблица 3. Порядок переходов показан на Рис. 3.

| **Фаза** | **Длительность** | **Особенности** |
|---|---|---|
| Красный | 4000 мс | Обычный период, в четыре раза больше зелёного |
| Красный по запросу | 1000 мс | От начала текущего или ближайшего красного |
| Зелёный | 1000 мс | Нажатия не изменяют цикл |
| Мигающий зелёный | 2000 мс | Восемь полупериодов по 250 мс; четыре интервала включения |
| Жёлтый | 1000 мс | После окончания следует красный |
| Подавление дребезга | 30 мс | Подтверждение устойчивого входного состояния |

Таблица 3. Временные параметры реализации




Рис. 3. Циклическая последовательность состояний светофора

Во время мигания значение `blink_edge` определяет состояние зелёного индикатора: при чётном значении он включён, при нечётном выключен. По достижении восьми границ полупериодов включается жёлтый. При входе в мигание зелёный уже горит после предыдущей фазы, поэтому граница фаз сама по себе не создаёт дополнительного выключения.

## 4.2. Обработка запроса пешехода

В структуре `TrafficLight` хранятся состояние, время входа в него, срок следующего действия, счётчик полупериодов и флаг `shorten_next_red`. Реакция на подтверждённое нажатие зависит от состояния:

- при постоянном зелёном событие игнорируется;
- при мигающем зелёном или жёлтом устанавливается флаг сокращения ближайшего красного, порядок фаз не меняется;
- при красном вычисляется срок `state_started_ms + TRAFFIC_SHORT_RED_MS`. Если этот срок уже достигнут, переход к зелёному выполняется сразу; иначе он заменяет обычный срок окончания красного.

При входе в красный флаг запроса определяет длительность и сразу сбрасывается. Таким образом, **один запрос влияет только на ближайший красный**, а следующий без нового запроса вновь длится 4000 мс. Несколько нажатий до

использования запроса не образуют очередь: они сохраняют один логический флаг.

Слово «сразу» означает обработку после подтверждения нажатия фильтром дребезга. Физическое замыкание контактов и событие программы разделены как минимум интервалом устойчивости 30 мс и временем до очередного опроса. Ветвление обработчика показано на Рис. 4.

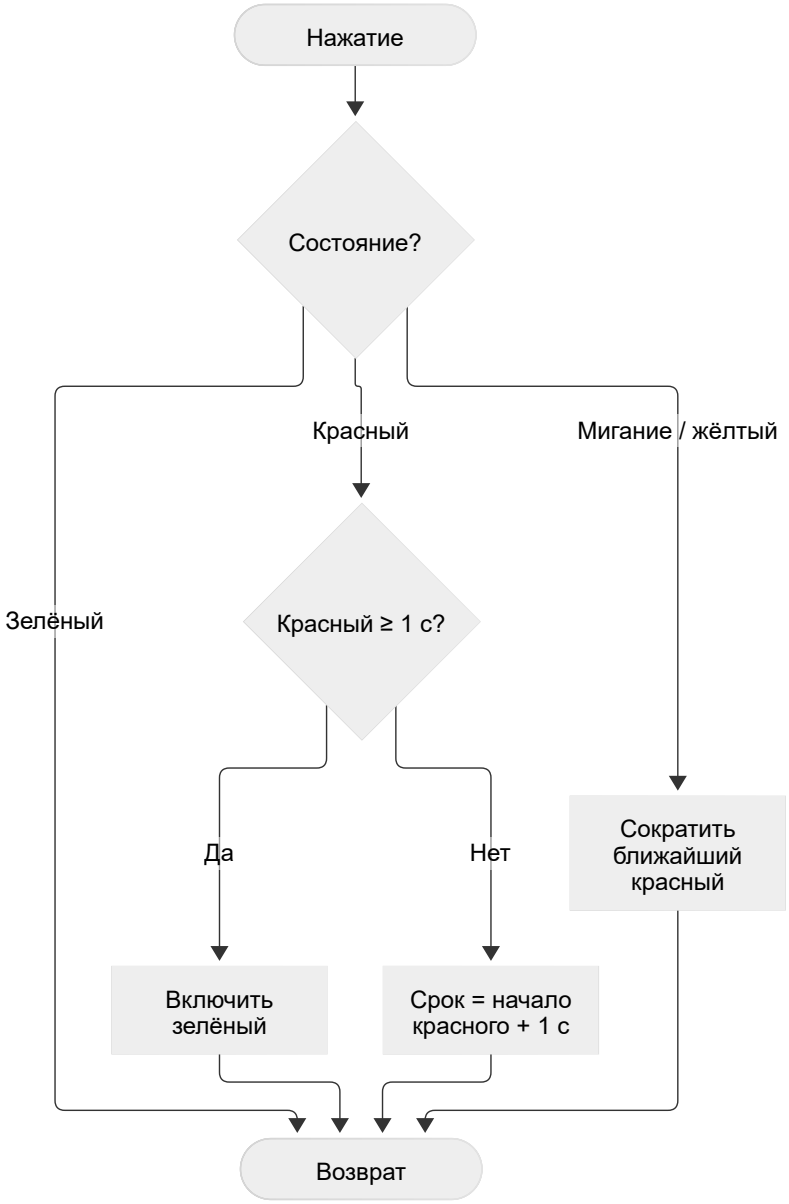



Рис. 4. Обработка подтверждённого нажатия кнопки

### 4.3. Подавление дребезга кнопки

Модуль кнопки хранит неподтверждённое состояние `candidate_pressed`, подтверждённое состояние `stable_pressed` и момент последнего изменения кандидата. Если считанный уровень отличается от кандидата, кандидат обновляется и отсчёт устойчивости начинается заново.

Состояние подтверждается только после 30 мс без изменений. Переход в нажатое состояние формирует `BUTTON_EVENT_PRESSED`, переход в отпущенное - `BUTTON_EVENT_RELEASED` с длительностью удержания. При удержании кнопки подтвер-

ждённое состояние не меняется, поэтому повторные события нажатия не возникают. В прикладной программе используется только событие нажатия.

При старте состояние кнопки считывается как начальное, поэтому уже зажатая при включении кнопка сама по себе не считается новым нажатием. Для появления события её требуется отпустить и нажать снова.

Интервалы дребезга и сроки переключений проверяются по разности показаний миллисекундного счётчика с учётом его переполнения; используемые интервалы меньше половины диапазона 32-разрядного счётчика.

# 5. Реализация собственного GPIO-драйвера

## 5.1. Интерфейс и регистры

Собственная структура `CustomGpioConfig` задаёт режим, тип выхода, скорость фронтов, подтяжку, альтернативную функцию и начальный выходной уровень. Настройка выполняется по маске выводов, чтение конфигурации - для одного вывода. Основные функции перечислены в Таблица 4.

| **Функция** | **Действие** |
|---|---|
| `custom_gpio_enable_clock` | Включение тактирования порта GPIOA-GPIOI |
| `custom_gpio_init` | Настройка группы выводов без изменения остальных |
| `custom_gpio_get_config` | Чтение режима и параметров заданного вывода |
| `custom_gpio_read_pin` | Чтение одного входного сигнала |
| `custom_gpio_read_mask` | Чтение выбранных входных сигналов |
| `custom_gpio_write_pin` | Изменение одного выходного сигнала |
| `custom_gpio_write_mask` | Установка и сброс выбранных выходов одной записью |

Таблица 4. Интерфейс собственного GPIO-драйвера

Драйвер обращается непосредственно к полям `GPIO_TypeDef`; функции `HAL_GPIO_*`, тип `GPIO_InitTypeDef` и функции LL не используются. Назначение регистров приведено в Таблица 5.

| **Регистр** | **Назначение в реализации** |
|---|---|
| `RCC->AHB1ENR` | Разрешение тактирования GPIO перед настройкой |
| `MODER` | По два бита на вывод: вход, выход, альтернативная функция или аналоговый режим |
| `OTYPER` | Выбор push-pull или open-drain |
| `OSPEEDR` | Настройка скорости переключения выхода |
| `PUPDR` | Отсутствие подтяжки, pull-up или pull-down |
| `IDR` | Чтение текущих уровней на входах |
| `ODR` | Чтение выходной защёлки при получении конфигурации |
| `BSRR` | Установка и сброс выходных битов без чтения и изменения `ODR` |
| `AFR[0/1]` | Выбор альтернативной функции для выводов 0-7 и 8-15 |

Таблица 5. Регистры, используемые драйвером

### 5.2. Групповая запись и порядок настройки

При записи группы выводов младшие 16 бит BSRR задают устанавливаемые биты, старшие 16 - сбрасываемые. Маски вычисляются так:

```
const uint32_t set_mask = (uint32_t)(pin_mask & high_mask);
const uint32_t reset_mask =
    (uint32_t)(pin_mask & (uint16_t)~high_mask) << 16U;
port->BSRR = set_mask | reset_mask;
```

Одна запись изменяет только выбранные выводы и позволяет одновременно переключить линии светодиодов. Атомарность относится к этой операции с выходными битами; вся последовательность настройки нескольких управляющих регистров одной атомарной операцией не является.

В `custom_gpio_init()` сначала проверяются параметры и включается тактирование. Для выходного и альтернативного режимов предварительно записывается требуемое значение через BSRR. Далее настраиваются тип выхода, скорость, подтяжка и альтернативная функция. Регистр MODER изменяется последним. Такой порядок предотвращает появление старого значения выходной защёлки при переходе из входного режима в обычный GPIO-выход.

Повторный вызов изменяет выбранную группу, сохраняя поля других выводов. Однако предварительная запись BSRR сама по себе **не доказывает отсутствие выбросов для любых переходов между режимами**, например при смене источника альтернативной функции или типа уже активного выхода. Для используемого перехода вход/выход предусмотрена предварительная загрузка уровня; универсальную гарантию следует проверять отдельно, в том числе измерением сигналов.

### 5.3. Отличия от HAL и LL

HAL предоставляет унифицированный интерфейс настройки и управления периферией, а LL - более близкие к регистрам операции. В разработанном варианте типы параметров, проверки и алгоритм записи полей определены в собственном модуле. Используется только описание аппаратных регистров из заголовков производителя.

Объём драйвера ограничен задачами лабораторной: поддерживаются обычные режимы GPIO и настройка альтернативной функции, но нет EXTI, обработки прерываний и блокировки конфигурации. HAL сохраняется для запуска микроконтроллера, настройки тактирования и системного времени; это не является использованием HAL для управления GPIO.

## 6. Сборка и загрузка программы

Проект `light` собирается в STM32CubeIDE для STM32F427VIT6. Прикладные модули размещены в `Core/Inc` и `Core/Src`, собственные фрагменты `main.c` - в секциях `USER CODE`; сгенерированные системные файлы сохранены. Полученный файл `Debug/light.elf` загружается встроенным отладчиком стенда через DBG USB с использованием OpenOCD и JTAG.

# 7. Проверка программы

## 7.1. Проверка на стенде

Ручная проверка работы светофора проведена на стенде SDK-1.1M наблюдением светодиодов и нажатием кнопки в разных фазах. В Таблица 6 приведены сценарии и ожидаемое поведение. Результаты работы светофора восстановлены по воспоминаниям участников проверки; подробный протокол и точные измерения длительностей не сохранились.

| № | Действия | Ожидаемый результат |
|---|---|---|
| 1 | Сбросить стенд и запустить без нажатий | Красный 4 с; зелёный 1 с; мигание 2 с; жёлтый 1 с; повтор цикла |
| 2 | Нажать в первые 0,5 с красного | Зелёный включается примерно через 1 с от начала красного |
| 3 | Нажать после 1,5 с красного | Зелёный включается после подтверждения нажатия |
| 4 | Нажать во время мигания | Мигание и жёлтый завершаются; ближайший красный длится 1 с |
| 5 | Нажать во время жёлтого | Жёлтый завершается; ближайший красный длится 1 с |
| 6 | Нажать во время постоянного зелёного | Длительности и порядок фаз не меняются |
| 7 | После сокращённого красного не нажимать кнопку | Следующий красный длится 4 с |
| 8 | Удерживать кнопку через несколько фаз | Нет повторных событий и остановки цикла |
| 9 | Повторно настроить GPIO; наблюдать сигнал прибором | Сохраняются остальные выводы; при включении выхода нет импульса старого уровня |

Таблица 6. Сценарии проверки на SDK-1.1M

**Фактические результаты ручной проверки:** сценарии 1-8 выполнены успешно; отклонений в работе светофора и реакции на кнопку не отмечено.

Без нажатий наблюдалась повторяющаяся последовательность: красный, зелёный, мигающий зелёный, жёлтый. Нажатие во время красного сокращало его период; при нажатии после истечения сокращённого периода выполнялся переход к зелёному. Нажатия во время мигания и жёлтого не нарушали порядок фаз и сокращали ближайший красный. Во время постоянного зелёного нажатия не изменяли работу светофора.

После использования запроса следующий красный без нового нажатия возвращался к обычной длительности. Удержание кнопки не вызывало повторных запросов и не останавливало цикл. Соответствие длительностей оценивалось качественно: численные значения в таблице являются заданными параметрами программы, а не результатами точных измерений.

Приборная проверка по сценарию 9 не подтверждена сохранившимися результатами. Визуальное наблюдение светодиодов не позволяет сделать вывод об отсутствии коротких выбросов GPIO.

## 8. Выводы

Разработана программа варианта 1 «Светофор». Конечный автомат сохраняет порядок фаз и сокращает ближайший красный период по запросу кнопки. Опрос и индикация выполняются кооперативно без блокирующих задержек; фильтр дребезга формирует однократное событие нажатия.

Разработан собственный GPIO-драйвер. Изучены настройка режима вывода, чтение входов и групповая запись через BSRR. Предварительная загрузка уровня до включения GPIO-выхода предотвращает использование старого значения выходной защёлки при таком переходе.

Ручная проверка на стенде подтвердила правильную последовательность цветов, обработку запросов кнопки и непрерывность цикла при её удержании. Отклонений в проверенных сценариях работы светофора не отмечено. Отсутствие коротких выбросов при перенастройке GPIO приборными измерениями не подтверждено.

## 9. Исходные коды собственной разработки

Ниже приведены собственные интерфейсы и реализации модулей. Текст листингов включается в документ при сборке и доступен для поиска. Сгенерированные системные файлы и исходники HAL/CMSIS не приводятся. Из `main.c` показана прикладная часть: настройка системной частоты остаётся в сгенерированной функции проекта `light`.

### 9.1. Основная программа: прикладная часть main.c

```
#include "main.h"
#include "button.h"
#include "sdk_board.h"
#include "traffic_light.h"

void SystemClock_Config(void);

static void set_traffic_lights(bool red, bool yellow, bool green)
{
    uint8_t mask = SDK_LED_NONE;
    mask |= red ? SDK_LED_RED : 0U;
    mask |= yellow ? SDK_LED_YELLOW : 0U;
    mask |= green ? SDK_LED_GREEN : 0U;
    sdk_board_write_leds(mask);
}

int main(void)
{
    Button button;
    TrafficLight traffic_light;

    HAL_Init();
    SystemClock_Config();
    sdk_board_init();

    const uint32_t now_ms = HAL_GetTick();
    button_init(&button, sdk_board_button_pressed(), now_ms, 30U);
    traffic_light_init(&traffic_light, now_ms, set_traffic_lights);

    while (1) {
        const uint32_t current_ms = HAL_GetTick();
        const ButtonEvent event = button_update(
            &button, sdk_board_button_pressed(), current_ms);

        if (event.type == BUTTON_EVENT_PRESSED) {
            traffic_light_on_button_pressed(&traffic_light, current_ms);
        }
        traffic_light_update(&traffic_light, current_ms);
    }
}
```

## 9.2. Интерфейс конечного автомата: traffic_light.h

```
#ifndef TRAFFIC_LIGHT_H
#define TRAFFIC_LIGHT_H

#include <stdbool.h>
#include <stdint.h>

enum {
    TRAFFIC_RED_MS = 4000U,
    TRAFFIC_GREEN_MS = 1000U,
    TRAFFIC_SHORT_RED_MS = TRAFFIC_RED_MS / 4U,
    TRAFFIC_BLINK_HALF_PERIOD_MS = 250U,
    TRAFFIC_BLINK_EDGE_COUNT = 8U,
    TRAFFIC_YELLOW_MS = 1000U
};

typedef enum {
    TRAFFIC_STATE_RED = 0,
    TRAFFIC_STATE_GREEN,
    TRAFFIC_STATE_BLINKING_GREEN,
    TRAFFIC_STATE_YELLOW
} TrafficLightState;

typedef void (*TrafficLightOutput)(bool red, bool yellow, bool green);

typedef struct {
    TrafficLightState state;
    uint32_t state_started_ms;
    uint32_t deadline_ms;
    uint8_t blink_edge;
    bool shorten_next_red;
    TrafficLightOutput output;
} TrafficLight;

void traffic_light_init(TrafficLight *traffic_light, uint32_t now_ms, TrafficLightOutput
output);
void traffic_light_on_button_pressed(TrafficLight *traffic_light, uint32_t now_ms);
void traffic_light_update(TrafficLight *traffic_light, uint32_t now_ms);

#endif
```

## 9.3. Реализация конечного автомата: traffic_light.c

```
#include "traffic_light.h"

static bool deadline_reached(uint32_t now_ms, uint32_t deadline_ms)
{
    return (int32_t)(now_ms - deadline_ms) >= 0;
}

static void apply_output(const TrafficLight *traffic_light)
{
    bool red = false;
    bool yellow = false;
    bool green = false;

    switch (traffic_light->state) {
        case TRAFFIC_STATE_RED:
            red = true;
            break;
        case TRAFFIC_STATE_GREEN:
            green = true;
            break;
        case TRAFFIC_STATE_BLINKING_GREEN:
            green = (traffic_light->blink_edge % 2U) == 0U;
            break;
        case TRAFFIC_STATE_YELLOW:
            yellow = true;
            break;
        default:
            break;
    }

    traffic_light->output(red, yellow, green);
}

static void enter_state(TrafficLight *traffic_light, TrafficLightState state, uint32_t now_ms)
{
    traffic_light->state = state;
    traffic_light->state_started_ms = now_ms;
    traffic_light->blink_edge = 0U;

    switch (state) {
        case TRAFFIC_STATE_RED:
            traffic_light->deadline_ms = now_ms
```

```
                + (traffic_light->shorten_next_red ? TRAFFIC_SHORT_RED_MS : TRAFFIC_RED_MS);
            traffic_light->shorten_next_red = false;
            break;
        case TRAFFIC_STATE_GREEN:
            traffic_light->deadline_ms = now_ms + TRAFFIC_GREEN_MS;
            break;
        case TRAFFIC_STATE_BLINKING_GREEN:
            traffic_light->deadline_ms = now_ms + TRAFFIC_BLINK_HALF_PERIOD_MS;
            break;
        case TRAFFIC_STATE_YELLOW:
            traffic_light->deadline_ms = now_ms + TRAFFIC_YELLOW_MS;
            break;
        default:
            break;
    }

    apply_output(traffic_light);
}

void traffic_light_init(TrafficLight *traffic_light, uint32_t now_ms, TrafficLightOutput
output)
{
    traffic_light->shorten_next_red = false;
    traffic_light->output = output;
    enter_state(traffic_light, TRAFFIC_STATE_RED, now_ms);
}

void traffic_light_on_button_pressed(TrafficLight *traffic_light, uint32_t now_ms)
{
    if (traffic_light->state == TRAFFIC_STATE_GREEN) {
        return;
    }

    if (traffic_light->state == TRAFFIC_STATE_RED) {
        const uint32_t shortened_deadline = traffic_light->state_started_ms +
TRAFFIC_SHORT_RED_MS;
        if (deadline_reached(now_ms, shortened_deadline)) {
            enter_state(traffic_light, TRAFFIC_STATE_GREEN, now_ms);
        } else {
            traffic_light->deadline_ms = shortened_deadline;
        }
    } else {
        traffic_light->shorten_next_red = true;
    }
}

void traffic_light_update(TrafficLight *traffic_light, uint32_t now_ms)
{
    for (uint32_t step = 0U; step < 16U; ++step) {
        if (!deadline_reached(now_ms, traffic_light->deadline_ms)) {
            return;
        }

        const uint32_t scheduled_ms = traffic_light->deadline_ms;
        switch (traffic_light->state) {
            case TRAFFIC_STATE_RED:
                enter_state(traffic_light, TRAFFIC_STATE_GREEN, scheduled_ms);
                break;
            case TRAFFIC_STATE_GREEN:
                enter_state(traffic_light, TRAFFIC_STATE_BLINKING_GREEN, scheduled_ms);
                break;
            case TRAFFIC_STATE_BLINKING_GREEN:
                ++traffic_light->blink_edge;
                if (traffic_light->blink_edge >= TRAFFIC_BLINK_EDGE_COUNT) {
                    enter_state(traffic_light, TRAFFIC_STATE_YELLOW, scheduled_ms);
                } else {
                    traffic_light->deadline_ms = scheduled_ms + TRAFFIC_BLINK_HALF_PERIOD_MS;
                    apply_output(traffic_light);
                }
                break;
            case TRAFFIC_STATE_YELLOW:
                enter_state(traffic_light, TRAFFIC_STATE_RED, scheduled_ms);
                break;
            default:
                enter_state(traffic_light, TRAFFIC_STATE_RED, now_ms);
                return;
        }
    }
}
```

## 9.4. Интерфейс кнопки: button.h

```
#ifndef BUTTON_H
#define BUTTON_H

#include <stdbool.h>
#include <stdint.h>

typedef enum {
    BUTTON_EVENT_NONE = 0,
    BUTTON_EVENT_PRESSED,
    BUTTON_EVENT_RELEASED
} ButtonEventType;

typedef struct {
    ButtonEventType type;
    uint32_t held_ms;
} ButtonEvent;

typedef struct {
    bool candidate_pressed;
    bool stable_pressed;
    uint32_t candidate_since_ms;
    uint32_t pressed_since_ms;
    uint32_t debounce_ms;
} Button;

void button_init(Button *button, bool initially_pressed, uint32_t now_ms, uint32_t
debounce_ms);
ButtonEvent button_update(Button *button, bool raw_pressed, uint32_t now_ms);

#endif
```

## 9.5. Подавление дребезга: button.c

```
#include "button.h"

static bool elapsed(uint32_t now_ms, uint32_t since_ms, uint32_t interval_ms)
{
    return (uint32_t)(now_ms - since_ms) >= interval_ms;
}

void button_init(Button *button, bool initially_pressed, uint32_t now_ms, uint32_t
debounce_ms)
{
    button->candidate_pressed = initially_pressed;
    button->stable_pressed = initially_pressed;
    button->candidate_since_ms = now_ms;
    button->pressed_since_ms = now_ms;
    button->debounce_ms = debounce_ms;
}

ButtonEvent button_update(Button *button, bool raw_pressed, uint32_t now_ms)
{
    ButtonEvent event = { .type = BUTTON_EVENT_NONE, .held_ms = 0U };

    if (raw_pressed != button->candidate_pressed) {
        button->candidate_pressed = raw_pressed;
        button->candidate_since_ms = now_ms;
        return event;
    }

    if ((button->stable_pressed == button->candidate_pressed)
        || !elapsed(now_ms, button->candidate_since_ms, button->debounce_ms)) {
        return event;
    }

    button->stable_pressed = button->candidate_pressed;
    if (button->stable_pressed) {
        button->pressed_since_ms = now_ms;
        event.type = BUTTON_EVENT_PRESSED;
    } else {
        event.type = BUTTON_EVENT_RELEASED;
        event.held_ms = (uint32_t)(now_ms - button->pressed_since_ms);
    }
    return event;
}
```

## 9.6. Интерфейс стенда: sdk_board.h

```
#ifndef SDK_BOARD_H
#define SDK_BOARD_H

#include <stdbool.h>
#include <stdint.h>

typedef enum {
    SDK_LED_NONE = 0U,
    SDK_LED_GREEN = 1U << 0,
    SDK_LED_YELLOW = 1U << 1,
    SDK_LED_RED = 1U << 2
} SdkLedMask;

void sdk_board_init(void);
bool sdk_board_button_pressed(void);
void sdk_board_write_leds(uint8_t led_mask);

#endif
```

## 9.7. Работа с элементами стенда: sdk_board.c

```
#include "sdk_board.h"

#include "custom_gpio.h"

enum {
    BUTTON_PIN = 15U,
    GREEN_LED_PIN = 13U,
    YELLOW_LED_PIN = 14U,
    RED_LED_PIN = 15U
};

static const uint16_t GREEN_LED_MASK = (uint16_t)(1U << GREEN_LED_PIN);
static const uint16_t YELLOW_LED_MASK = (uint16_t)(1U << YELLOW_LED_PIN);
static const uint16_t RED_LED_MASK = (uint16_t)(1U << RED_LED_PIN);
static const uint16_t ALL_LED_MASK = (uint16_t)(
    (1U << GREEN_LED_PIN) | (1U << YELLOW_LED_PIN) | (1U << RED_LED_PIN));

void sdk_board_init(void)
{
    const CustomGpioConfig led_config = {
        .mode = CUSTOM_GPIO_MODE_OUTPUT,
        .output_type = CUSTOM_GPIO_PUSH_PULL,
        .speed = CUSTOM_GPIO_SPEED_LOW,
        .pull = CUSTOM_GPIO_NO_PULL,
        .alternate_function = 0U,
        .output_high = false
    };
    const CustomGpioConfig button_config = {
        .mode = CUSTOM_GPIO_MODE_INPUT,
        .output_type = CUSTOM_GPIO_PUSH_PULL,
        .speed = CUSTOM_GPIO_SPEED_LOW,
        .pull = CUSTOM_GPIO_NO_PULL,
        .alternate_function = 0U,
        .output_high = false
    };

    (void)custom_gpio_init(GPIOD, ALL_LED_MASK, &led_config);
    (void)custom_gpio_init(GPIOC, (uint16_t)(1U << BUTTON_PIN), &button_config);
}

bool sdk_board_button_pressed(void)
{
    return !custom_gpio_read_pin(GPIOC, BUTTON_PIN);
}

void sdk_board_write_leds(uint8_t led_mask)
{
    uint16_t high_mask = 0U;

    if ((led_mask & SDK_LED_GREEN) != 0U) {
        high_mask |= GREEN_LED_MASK;
    }

    const bool red = (led_mask & SDK_LED_RED) != 0U;
    const bool yellow = (led_mask & SDK_LED_YELLOW) != 0U;
    if (red && !yellow) {
        high_mask |= RED_LED_MASK;
    } else if (yellow && !red) {
        high_mask |= YELLOW_LED_MASK;
    }

    custom_gpio_write_mask(GPIOD, ALL_LED_MASK, high_mask);
}
```

## 9.8. Интерфейс GPIO-драйвера: custom_gpio.h

```
#ifndef CUSTOM_GPIO_H
#define CUSTOM_GPIO_H

#include <stdbool.h>
#include <stdint.h>

#include "stm32f4xx.h"

typedef enum {
    CUSTOM_GPIO_MODE_INPUT = 0U,
    CUSTOM_GPIO_MODE_OUTPUT = 1U,
    CUSTOM_GPIO_MODE_ALTERNATE = 2U,
    CUSTOM_GPIO_MODE_ANALOG = 3U
} CustomGpioMode;

typedef enum {
    CUSTOM_GPIO_PUSH_PULL = 0U,
    CUSTOM_GPIO_OPEN_DRAIN = 1U
} CustomGpioOutputType;

typedef enum {
    CUSTOM_GPIO_SPEED_LOW = 0U,
    CUSTOM_GPIO_SPEED_MEDIUM = 1U,
    CUSTOM_GPIO_SPEED_HIGH = 2U,
    CUSTOM_GPIO_SPEED_VERY_HIGH = 3U
} CustomGpioSpeed;

typedef enum {
    CUSTOM_GPIO_NO_PULL = 0U,
    CUSTOM_GPIO_PULL_UP = 1U,
    CUSTOM_GPIO_PULL_DOWN = 2U
} CustomGpioPull;

typedef struct {
    CustomGpioMode mode;
    CustomGpioOutputType output_type;
    CustomGpioSpeed speed;
    CustomGpioPull pull;
    uint8_t alternate_function;
    bool output_high;
} CustomGpioConfig;

bool custom_gpio_enable_clock(GPIO_TypeDef *port);
bool custom_gpio_init(GPIO_TypeDef *port, uint16_t pin_mask, const CustomGpioConfig *config);
bool custom_gpio_get_config(const GPIO_TypeDef *port, uint8_t pin, CustomGpioConfig *config);

bool custom_gpio_read_pin(const GPIO_TypeDef *port, uint8_t pin);
uint16_t custom_gpio_read_mask(const GPIO_TypeDef *port, uint16_t pin_mask);
void custom_gpio_write_pin(GPIO_TypeDef *port, uint8_t pin, bool high);
void custom_gpio_write_mask(GPIO_TypeDef *port, uint16_t pin_mask, uint16_t high_mask);

#endif
```

## 9.9. Реализация GPIO-драйвера: custom_gpio.c

```
#include "custom_gpio.h"

static bool is_valid_pin(uint8_t pin)
{
    return pin < 16U;
}

static uint32_t field_mask_for_pins(uint16_t pin_mask)
{
    uint32_t field_mask = 0U;

    for (uint32_t pin = 0U; pin < 16U; ++pin) {
        if ((pin_mask & (uint16_t)(1U << pin)) != 0U) {
            field_mask |= 3UL << (pin * 2U);
        }
    }
    return field_mask;
}

static uint32_t field_value_for_pins(uint16_t pin_mask, uint32_t value)
{
    uint32_t field_value = 0U;

    for (uint32_t pin = 0U; pin < 16U; ++pin) {
        if ((pin_mask & (uint16_t)(1U << pin)) != 0U) {
            field_value |= value << (pin * 2U);
        }
    }
    return field_value;
```

```
}

bool custom_gpio_enable_clock(GPIO_TypeDef *port)
{
    uint32_t enable_mask;

    if (port == GPIOA) {
        enable_mask = RCC_AHB1ENR_GPIOAEN;
    } else if (port == GPIOB) {
        enable_mask = RCC_AHB1ENR_GPIOBEN;
    } else if (port == GPIOC) {
        enable_mask = RCC_AHB1ENR_GPIOCEN;
    } else if (port == GPIOD) {
        enable_mask = RCC_AHB1ENR_GPIODEN;
    } else if (port == GPIOE) {
        enable_mask = RCC_AHB1ENR_GPIOEEN;
    } else if (port == GPIOF) {
        enable_mask = RCC_AHB1ENR_GPIOFEN;
    } else if (port == GPIOG) {
        enable_mask = RCC_AHB1ENR_GPIOGEN;
    } else if (port == GPIOH) {
        enable_mask = RCC_AHB1ENR_GPIOHEN;
    } else if (port == GPIOI) {
        enable_mask = RCC_AHB1ENR_GPIOIEN;
    } else {
        return false;
    }

    RCC->AHB1ENR |= enable_mask;
    (void)RCC->AHB1ENR;
    return true;
}

bool custom_gpio_init(GPIO_TypeDef *port, uint16_t pin_mask, const CustomGpioConfig *config)
{
    if ((port == NULL) || (config == NULL) || (pin_mask == 0U)
        || ((uint32_t)config->mode > (uint32_t)CUSTOM_GPIO_MODE_ANALOG)
        || ((uint32_t)config->output_type > (uint32_t)CUSTOM_GPIO_OPEN_DRAIN)
        || ((uint32_t)config->speed > (uint32_t)CUSTOM_GPIO_SPEED_VERY_HIGH)
        || ((uint32_t)config->pull > (uint32_t)CUSTOM_GPIO_PULL_DOWN)
        || (config->alternate_function > 15U)) {
        return false;
    }

    if (!custom_gpio_enable_clock(port)) {
        return false;
    }

    if ((config->mode == CUSTOM_GPIO_MODE_OUTPUT)
        || (config->mode == CUSTOM_GPIO_MODE_ALTERNATE)) {
        custom_gpio_write_mask(port, pin_mask, config->output_high ? pin_mask : 0U);
    }

    const uint32_t two_bit_mask = field_mask_for_pins(pin_mask);

    port->OTYPER = (port->OTYPER & ~(uint32_t)pin_mask)
        | (config->output_type == CUSTOM_GPIO_OPEN_DRAIN ? pin_mask : 0U);
    port->OSPEEDR = (port->OSPEEDR & ~two_bit_mask)
        | field_value_for_pins(pin_mask, (uint32_t)config->speed);
    port->PUPDR = (port->PUPDR & ~two_bit_mask)
        | field_value_for_pins(pin_mask, (uint32_t)config->pull);

    for (uint32_t pin = 0U; pin < 16U; ++pin) {
        if ((pin_mask & (uint16_t)(1U << pin)) != 0U) {
            const uint32_t index = pin / 8U;
            const uint32_t shift = (pin % 8U) * 4U;
            port->AFR[index] = (port->AFR[index] & ~(0xFUL << shift))
                | ((uint32_t)config->alternate_function << shift);
        }
    }

    port->MODER = (port->MODER & ~two_bit_mask)
        | field_value_for_pins(pin_mask, (uint32_t)config->mode);
    return true;
}

bool custom_gpio_get_config(const GPIO_TypeDef *port, uint8_t pin, CustomGpioConfig *config)
{
    if ((port == NULL) || (config == NULL) || !is_valid_pin(pin)) {
        return false;
    }

    const uint32_t shift = (uint32_t)pin * 2U;
    const uint32_t af_shift = ((uint32_t)pin % 8U) * 4U;

    config->mode = (CustomGpioMode)((port->MODER >> shift) & 3U);
    config->output_type = (CustomGpioOutputType)((port->OTYPER >> pin) & 1U);
    config->speed = (CustomGpioSpeed)((port->OSPEEDR >> shift) & 3U);
    config->pull = (CustomGpioPull)((port->PUPDR >> shift) & 3U);
    config->alternate_function = (uint8_t)((port->AFR[pin / 8U] >> af_shift) & 0xFU);
```

```
    config->output_high = ((port->ODR >> pin) & 1U) != 0U;
    return true;
}

bool custom_gpio_read_pin(const GPIO_TypeDef *port, uint8_t pin)
{
    return (port != NULL) && is_valid_pin(pin) && (((port->IDR >> pin) & 1U) != 0U);
}

uint16_t custom_gpio_read_mask(const GPIO_TypeDef *port, uint16_t pin_mask)
{
    return port == NULL ? 0U : (uint16_t)(port->IDR & pin_mask);
}

void custom_gpio_write_pin(GPIO_TypeDef *port, uint8_t pin, bool high)
{
    if ((port != NULL) && is_valid_pin(pin)) {
        const uint16_t pin_mask = (uint16_t)(1U << pin);
        custom_gpio_write_mask(port, pin_mask, high ? pin_mask : 0U);
    }
}

void custom_gpio_write_mask(GPIO_TypeDef *port, uint16_t pin_mask, uint16_t high_mask)
{
    if (port == NULL) {
        return;
    }

    const uint32_t set_mask = (uint32_t)(pin_mask & high_mask);
    const uint32_t reset_mask = (uint32_t)(pin_mask & (uint16_t)~high_mask) << 16U;
    port->BSRR = set_mask | reset_mask;
}
```